Fundamentos de Programación

Cómo usar esta guía

Esta guía ordena tus apuntes de Python en una sola ruta de estudio: primero cómo se construye un programa, luego cómo se organiza y protege el código, y al final las estructuras de datos y los errores típicos.

Tus apuntes originales venían en tres bloques (clases y objetos; control de flujo, excepciones y f-strings; repaso de arreglos, fechas y diccionarios). Aquí se fusionaron por tema para que no tengas que saltar entre ellos.

Ruta de estudio

1. Ciclo de desarrollo de un programa
2. Clases y objetos: atributos, métodos, `self` y constructor
3. Type hints
4. Validaciones y excepciones: `raise`, `try/except`
5. Estructuras de control: `input()`, `break`, `continue`, `return`, `while True`
6. Colecciones: listas, arreglos y diccionarios
7. Fechas con la clase `date`
8. Separación entre lógica e interfaz
9. Formato de texto con f-strings
10. Guía para diseñar una clase
11. Errores comunes y cómo corregirlos
12. Conceptos clave, glosario y preguntas de repaso

Nota sobre el código. En el PDF original todo el texto estaba en minúsculas, incluido el código (`typeerror`, `true`, `none`). Aquí se restauró la sintaxis válida de Python (`TypeError`, `True`, `None`) y se corrigieron unos pocos errores de indentación, para que puedas copiar y ejecutar los ejemplos.

Cómo estudiar con ella. Lee una sección, ejecuta sus ejemplos en tu editor y luego intenta responder las preguntas de repaso del final sin mirar.

1. Ciclo de desarrollo de un programa

Todo software profesional se construye en cuatro etapas, en este orden: definir requisitos, diseñar, codificar y probar.

Etapa	Qué se hace	Ejemplo aplicado
1. Definir requisitos	Determinar qué necesita el usuario	La empresa necesita gestionar empleados
2. Diseñar la solución	Definir clases, métodos y atributos necesarios	Se diseña una clase `Empleado` con sus atributos
3. Codificar	Escribir el código en Python	Se escribe la clase y sus métodos
4. Probar	Verificar que el programa funciona correctamente	Se crean empleados de ejemplo para validar

Por qué cada etapa es crucial

- **Requisitos:** sin saber qué se necesita, no se puede construir nada útil.
- **Diseño:** planificar antes de codificar evita errores costosos.
- **Codificación:** se implementa siguiendo el diseño.
- **Prueba:** se comprueba que lo construido realmente funciona.

2. Clases y objetos

Una clase es la plantilla que define cómo es y qué hace un objeto; cada objeto es una instancia de esa plantilla con sus propios valores.

2.1 Clase y objeto

- **Clase:** plantilla o molde que define la estructura y el comportamiento de los objetos.
- **Objeto:** instancia concreta de una clase, con valores específicos en sus atributos.

```
class Empleado:
    # cuerpo de la clase
    pass
```

Para qué sirven las clases:

- Representan entidades del mundo real (empleado, producto, factura).
- Definen **atributos**: características o datos del objeto.
- Definen **métodos**: acciones que puede realizar el objeto.
- Sirven como plano para crear muchos objetos similares.

2.2 Atributos

Los atributos son variables que pertenecen al objeto y guardan sus datos.

```
class Empleado:
    def __init__(self):
        self.nombre = ""      # atributo de tipo str
        self.edad = 0         # atributo de tipo int
        self.salario = 0.0    # atributo de tipo float
```

Características de los atributos:

- Se crean normalmente en el método `__init__`.
- Cada objeto tiene sus propios valores.
- Se acceden con la notación `objeto.atributo`.
- Pueden modificarse directamente o a través de métodos.

Tipos de datos comunes para atributos:

Tipo	Descripción	Ejemplo
`str`	Cadena de texto	`"Laura Gómez"`
`int`	Número entero	`25`
`float`	Número decimal	`2500000.50`
`date`	Fecha	`date(1995, 3, 12)`
`list`	Lista de elementos	`[1200000, 1500000]`
`bool`	Verdadero o falso	`True`

2.3 Métodos

Los métodos son funciones que pertenecen a una clase y definen el comportamiento de sus objetos.

```
class Empleado:
    def calcular_edad(self, fecha_referencia):
        # código del método
        return edad
```

Partes de un método:

Elemento	Descripción
`def`	Palabra clave para definir el método
Nombre	Identificador del método

Elemento	Descripción
`(self, ...)`	Parámetros; siempre incluye `self`
`return`	Devuelve un resultado (opcional)

2.4 El parámetro `self`

`self` es el objeto mismo: la referencia al objeto que se está creando o usando en ese momento.

- Es el primer parámetro de todo método.
- Permite acceder a los atributos del objeto.
- Python lo pasa automáticamente al llamar al método.

```
# definición
def modificar_salario(self, nuevo_salario):
    self.salario = nuevo_salario

# llamada: Python pasa "laura" automáticamente como self
laura.modificar_salario(2500000)
```

2.5 Qué puede usar un método

Regla mental: un método combina cinco ingredientes para hacer cálculos y acciones.

Componente	Ejemplo	Descripción
Atributos	`self.base`	Datos propios del objeto
Parámetros	`base`	Valores que entran al método
Constantes	`DIAS_POR_AÑO = 365`	Valores fijos definidos en el código
Variables internas	`area`	Viven solo dentro del método
Otros métodos	`self.calcular_edad()`	Llamadas a otros métodos del mismo objeto

2.6 Ejemplo de método completo

Este método usa los cinco ingredientes: una constante, una variable interna, atributos, un parámetro y otro método.

```
def calcular_area_total(self, factor_extra: float) -> float:
    # constante
    descuento_base = 0.95

    # variable interna (usa atributos del objeto)
    area_base = self.base * self.altura

    # usa parámetro y constante
    area_ajustada = area_base * factor_extra * descuento_base

    # llama a otro método
    if self.validar_area(area_ajustada):
        return area_ajustada
    else:
        return 0.0
```

2.7 El constructor `__init__`

`__init__` es el constructor: se ejecuta automáticamente cada vez que se crea un objeto.

```
class Empleado:
    def __init__(self,
                 primer_nombre: str,
                 segundo_nombre: str,
                 primer_apellido: str,
                 segundo_apellido: str,
                 fecha_nacimiento: date,
                 fecha_ingreso: date,
                 salario_base: float):
        # cada parámetro se guarda como atributo
        self.primer_nombre = primer_nombre
        self.segundo_nombre = segundo_nombre
        self.primer_apellido = primer_apellido
        self.segundo_apellido = segundo_apellido
        self.fecha_nacimiento = fecha_nacimiento
        self.fecha_ingreso = fecha_ingreso
        self.salario_base = salario_base
        # atributo adicional que no viene de un parámetro
        self.historial_salarios = []
```

Para qué sirve el constructor:

- Inicializar los atributos del objeto.
- Validar los datos recibidos.
- Crear atributos adicionales (listas, valores calculados).

- Preparar el objeto para su uso.

2.8 Creación de objetos

Crear un objeto propio es igual de simple que crear un tipo incorporado: se llama a la clase como si fuera una función.

```
# creación de tipos básicos
cantidad = int("6")          # entero
precio = float("50.99")      # flotante
nombre = str("Jean")         # string
fecha = date(1979, 3, 22)    # fecha

# creación de un objeto propio
laura = Empleado("Laura", "Isabel", "Gómez", "Torres",
                 date(1995, 3, 12), date(2022, 8, 5), 1900000)
```

Qué pasa al ejecutar `Empleado(...)`:

1. Se llama a la clase como si fuera una función.
2. Python crea un nuevo objeto en memoria.
3. Ejecuta automáticamente `__init__`.
4. Retorna el objeto creado.
5. El objeto se asigna a una variable (`laura`).

En el PDF original la variable se llamaba igual que la clase (`empleado = empleado(...)`). Usa nombres distintos: si la variable tiene el nombre de la clase, la clase queda tapada y ya no puedes crear más objetos.

3. Type hints

Los type hints son anotaciones que documentan qué tipo de dato se espera en parámetros, atributos y retornos; Python no los hace cumplir.

```
def __init__(self,
             primer_nombre: str,          # espera un string
             fecha_nacimiento: date,      # espera una fecha
             salario_base: float) -> None:  # no retorna nada
    self.primer_nombre: str = primer_nombre
```

Característica	Qué significa
No fuerzan tipos	Python no valida que el dato sea del tipo indicado

Característica	Qué significa
No previenen errores	Si pasas un tipo incorrecto, el programa se ejecuta igual
No cambian la ejecución	El código corre igual con o sin type hints
Aclaran el código	Documentan las intenciones del programador
Ayudan a los IDE	Permiten autocompletado y detección temprana de errores

Uso correcto en una clase

```
from datetime import date

class Empleado:
    def __init__(self,
                 nombre: str,              # el hint documenta
                 fecha_nacimiento: date,   # que se espera una fecha
                 salario: float) -> None:  # y que no retorna nada
        self.nombre: str = nombre
        self.fecha_nacimiento: date = fecha_nacimiento
        self.salario: float = salario
        self.historial: list[float] = []

    def calcular_edad(self, fecha_referencia: date) -> int:
        # retorna un entero
        return edad
```

Si necesitas que el tipo se cumpla de verdad, un type hint no basta: hay que validarlo con `isinstance` y `raise` (sección 4).

4. Validaciones y excepciones

`raise` detiene la ejecución de forma controlada cuando un dato incumple las reglas, y `try/except` captura ese error para manejarlo sin que el programa se caiga.

4.1 Validar con `raise`

Las validaciones aseguran que los datos cumplan las reglas del negocio antes de ser procesados. Se escriben al inicio del constructor.

Excepción	Cuándo usarla	Ejemplo
`TypeError`	El tipo de dato es incorrecto	`"5" + 10`
`ValueError`	El tipo es correcto, pero el valor no es apropiado (negativo, fecha futura, etc.)	`int("hola")`

```
class Gasto:
    def __init__(self, descripcion: str, monto: float, fecha: date):
        # validación de tipo
        if not isinstance(monto, (int, float)):
            raise TypeError("El monto debe ser un número")

        # validación de valor positivo
        if monto <= 0:
            raise ValueError("El monto debe ser positivo")

        # validación de tipo de fecha
        if not isinstance(fecha, date):
            raise TypeError("La fecha debe ser un objeto date")

        # validación de fecha no futura
        if fecha > date.today():
            raise ValueError("La fecha no puede ser futura")

        # si todo está bien, se asignan los atributos
        self.descripcion = descripcion
        self.monto = monto
        self.fecha = fecha
```

4.2 Capturar errores con `try/except`

Las excepciones se capturan en el programa principal (la interfaz), no dentro de la clase.

```
try:
    gasto = Gasto("Almuerzo", -15000, date.today())
except ValueError as e:
    print(f"Error: {e}")
except TypeError as e:
    print(f"Error de tipo: {e}")
```

4.3 La palabra clave `as`

`as` asigna el objeto de la excepción a una variable, que guarda el mensaje exacto y el tipo del error.

```
try:
    numero = int(input("Ingrese un número: "))
    resultado = 10 / numero
except ValueError as error:
    print(f"Error de valor: {error}")      # "invalid literal for int()..."
except ZeroDivisionError as error:
    print(f"Error de división: {error}")   # "division by zero"
```

4.4 `TypeError` frente a `ValueError`

```
try:
    resultado = "5" + 10          # str + int no es válido
except TypeError as error:
    print(f"TypeError: {error}")  # solo se puede concatenar str con str

try:
    numero = int("hola")          # "hola" no se puede convertir a entero
except ValueError as error:
    print(f"ValueError: {error}") # invalid literal for int() with base 10:
'hola'
```

4.5 Flujo con `try-except-return`

Si ocurre una excepción y el `except` ejecuta `return`, la función termina ahí y el código posterior no se ejecuta. Si no hay excepción, el flujo sigue después del bloque.

```
def registrar_empleado(datos):
    try:
        nuevo = Empleado(datos["nombre"], datos["salario"])
        print("Empleado creado exitosamente")
    except (TypeError, ValueError) as error:
        print(f"Error al crear empleado: {error}")
        return None   # la función termina aquí

    # este código solo se ejecuta si no hubo error
    print("Guardando en base de datos...")
    guardar_empleado(nuevo)
    return nuevo
```

5. Estructuras de control en funciones

`break` sale de un bucle, `continue` salta a la siguiente vuelta, `return` sale de la función completa, y `while True` con `return` es el patrón estándar para pedir un dato hasta que sea válido.

5.1 `input()` como pausa

`input()` detiene el programa y espera a que el usuario presione Enter. No es un bucle: es una pausa lineal. Al presionar Enter, el programa sigue con la siguiente línea o termina la función si no hay más.

```
def mostrar_datos_empleado(empleado):
    print(f"Nombre: {empleado.nombre}")
    print(f"Salario: {empleado.salario}")
    input("\nPresione Enter para continuar")   # el programa se pausa aquí
    # al presionar Enter, la función termina y vuelve al menú principal
```

5.2 Diferencia entre `break`, `continue` y `return`

Palabra	Dónde se usa	Qué hace
`break`	Solo dentro de bucles	Sale del bucle de inmediato
`continue`	Solo dentro de bucles	Salta a la siguiente iteración, omitiendo el código restante
`return`	Dentro de funciones	Sale de la función y devuelve un valor; termina también cualquier bucle en el que esté

```
# break: el bucle termina cuando i llega a 5
for i in range(10):
    if i == 5:
        break
    print(i)        # imprime 0, 1, 2, 3, 4

# continue: se salta solo la vuelta en que i vale 2
for i in range(5):
    if i == 2:
        continue
    print(i)        # imprime 0, 1, 3, 4

# return: termina la función y devuelve el empleado
def buscar_empleado(id):
    for empleado in lista_empleados:
        if empleado.id == id:
            return empleado
    return None     # si no lo encuentra, devuelve None
```

5.3 Bucle `while True` con validación

En una función que pide datos hasta que sean válidos, `while True` crea un bucle que no termina por sí solo. Solo cuando el dato es correcto se ejecuta `return`, que termina la función y con ella el bucle. Si el dato es incorrecto, se llega al final del bloque y comienza una nueva vuelta automáticamente, sin necesidad de `continue`.

```
def pedir_entero_en_rango(minimo, maximo):
    while True:
        try:
            valor = int(input(f"Ingrese un número ({minimo}-{maximo}): "))
            if minimo <= valor <= maximo:
                return valor        # único punto de salida, cuando es válido
            print("Error: valor fuera de rango")
        except ValueError:
            print("Error: debe digitar un número entero")
        # al llegar aquí, vuelve al inicio del while automáticamente
```

6. Colecciones: listas, arreglos y diccionarios

Una colección guarda varios datos juntos: la lista es la más flexible, el arreglo exige un solo tipo de dato y el diccionario asocia claves con valores.

6.1 Listas

Una lista almacena varios elementos en secuencia. Cada elemento tiene un índice, y el primero es el 0.

Índice	0	1	2
`salarios`	1200000.0	1500000.0	2500000.0

```
# crear lista
salarios = [1200000.0, 1500000.0, 2500000.0]

# acceder a elementos
primer_salario = salarios[0]     # 1200000.0
ultimo_salario = salarios[2]     # 2500000.0

# modificar un elemento
salarios[1] = 1600000.0

# agregar un elemento al final
salarios.append(3000000.0)

# longitud de la lista
cantidad = len(salarios)         # 4
```

Concepto	Fórmula	Ejemplo
Primer elemento	`lista[0]`	`salarios[0]`
Último elemento	`lista[len(lista)-1]`	`salarios[len(salarios)-1]`
Recorrer la lista	`for elemento in lista:`	`for salario in salarios:`
Recorrer con índice	`for i in range(len(lista)):`	`for i in range(len(salarios)):`

6.2 Listas que se llenan «solas»

Una lista que se llena sola es un atributo que guarda automáticamente valores históricos cada vez que se ejecuta cierto método. Es la «memoria» del objeto.

```
class Empleado:
    def __init__(self, salario_inicial):
        self.salario_actual = salario_inicial
        self.historial_salarios = []          # lista vacía inicial

    def cambiar_salario(self, nuevo_salario):
        # 1. guardar el valor actual antes de cambiarlo
        self.historial_salarios.append(self.salario_actual)
        # 2. actualizar al nuevo valor
        self.salario_actual = nuevo_salario
```

Qué pasa en memoria, paso a paso:

Paso	Acción	`salario_actual`	`historial_salarios`
1	Crear empleado	1900000	`[]`
2	`cambiar_salario(2100000)`	2100000	`[1900000]`
3	`cambiar_salario(2500000)`	2500000	`[1900000, 2100000]`

Aplicaciones comunes: historial de cambios de salario, registro de operaciones realizadas, seguimiento de versiones y bitácora de eventos.

6.3 Arreglos (`array`)

Un arreglo almacena varios valores **del mismo tipo** en posiciones contiguas de memoria, una al lado de otra y sin huecos.

Características:

- Todos los elementos deben ser del mismo tipo.
- Los elementos ocupan posiciones contiguas en memoria.
- El acceso por índice es muy eficiente.
- En Python el tamaño puede crecer con `append` , pero conceptualmente es fijo.

	Lista (`list`)	Arreglo (`array`)
Tipos de dato	Puede mezclar tipos	Un solo tipo declarado
Al crearla	No se especifica el tipo	Se especifica con un código de tipo
Seguridad	Máxima flexibilidad	Seguridad de tipos: no permite mezclar

Códigos de tipo: `'i'` para enteros (`int`) y `'d'` para decimales de doble precisión (`float`).

```
from array import array

# crear arreglos vacíos
edades = array('i', [])
temperaturas = array('d', [])

# crear con valores iniciales
notas = array('d', [3.5, 4.0, 2.8, 4.5])

notas.append(3.2)           # agregar un elemento
print(notas[0])             # acceder: primer elemento
print(notas[3])             # acceder: cuarto elemento
notas[2] = 3.0              # modificar un valor

for elemento in notas:      # recorrer
    print(elemento)

for i in range(len(notas)):  # recorrer con índice
    print(f"Posición {i}: {notas[i]}")

if 4.0 in notas:            # buscar un valor
    print("Está")

print(len(notas))           # longitud
```

6.4 Cálculos con arreglos: total, promedio, mayor y menor

Los cálculos siguen siempre el mismo patrón: un acumulador que se inicializa antes del bucle y se actualiza en cada vuelta.

```
from array import array

notas = array('d', [3.5, 4.0, 2.8, 4.5])

# total (acumulador)
total = 0.0
for nota in notas:
    total = total + nota

# promedio (verificar antes que no esté vacío)
if len(notas) > 0:
    promedio = total / len(notas)
```

```
# mayor y menor (inicializar con el primer elemento)
mayor = notas[0]
menor = notas[0]
for nota in notas:
    if nota > mayor:
        mayor = nota
    if nota < menor:
        menor = nota

print(f"Mayor: {mayor}, menor: {menor}")
```

Importante: inicializa `mayor` y `menor` con el primer elemento del arreglo, nunca con 0. Si todos los valores son negativos, empezar en 0 da un resultado incorrecto.

```
temperaturas = array('d', [-2.5, -5.0, -1.8, -3.2, -4.1])

# incorrecto: imprime 0, pero debería ser -1.8
mayor = 0
for t in temperaturas:
    if t > mayor:
        mayor = t
print(mayor)

# correcto: imprime -1.8
mayor = temperaturas[0]
for t in temperaturas:
    if t > mayor:
        mayor = t
print(mayor)
```

6.5 Diccionarios

Un diccionario guarda pares clave → valor, y se consulta por la clave en lugar de por una posición numérica. Es ideal para contar cosas por categoría.

Ventaja frente a una lista: el código se lee solo.

- Con diccionario, `dias_por_mes[3]` significa directamente «días de marzo».
- Con lista, `lista[2]` también es marzo, pero hay que recordar que el índice 0 es enero.

```
# diccionario: la clave es el número del mes
dias_por_mes = {1: 0, 2: 0, 3: 0}
print(dias_por_mes[3])      # claramente es marzo

# lista equivalente: ¿qué mes es el índice 2? hay que recordar que 0 = enero
dias_por_mes_lista = [0, 0, 0]
print(dias_por_mes_lista[2])
```

6.6 El método `get()`

Acceder con corchetes a una clave que no existe produce un `KeyError`. `get()` evita ese error y devuelve un valor por defecto.

```
conteo = {1: 5, 2: 3}

print(conteo[1])         # 5
# print(conteo[3])       # KeyError: la clave 3 no existe

valor = conteo.get(3, 0) # sintaxis: diccionario.get(clave,
valor_por_defecto)
```

Llamada	Si la clave existe	Si la clave no existe
`conteo.get(1)`	Devuelve su valor (`5`)	Devuelve `None` (sin error)
`conteo.get(3, 0)`	Devuelve su valor	Devuelve el valor por defecto (`0`)

Patrón común para contar: `get()` reemplaza el `if/else` de tres líneas por una sola.

```
# en lugar de:
if clave in diccionario:
    diccionario[clave] = diccionario[clave] + 1
else:
    diccionario[clave] = 1

# se puede usar:
diccionario[clave] = diccionario.get(clave, 0) + 1
```

7. Fechas con la clase `date`

Un objeto `date` representa una fecha con año, mes y día; se compara por valor y se filtra siempre por año **y** mes.

7.1 Crear y usar fechas

```
from datetime import date

# crear fechas específicas
fecha1 = date(2024, 3, 15)
fecha2 = date(2024, 12, 25)

# fecha actual
hoy = date.today()

# acceder a los atributos
print(fecha1.year)    # 2024
print(fecha1.month)   # 3
print(fecha1.day)     # 15

# comparar fechas
if fecha1 < fecha2:
    print("fecha1 es anterior")
```

- Un objeto `date` no es una cadena de texto: es un objeto con atributos numéricos.
- Se puede comparar con `<`, `>` y `==`.
- Dos fechas con el mismo año, mes y día son iguales: la comparación es **por valor**, no por identidad.

```
fecha_a = date(2024, 5, 10)
fecha_b = date(2024, 5, 10)
print(fecha_a == fecha_b)   # True, aunque son objetos diferentes
```

7.2 Listas de fechas

Como `date` se compara por valor, el operador `in` funciona para buscar una fecha dentro de una lista.

```
fechas: list[date] = []

fechas.append(date(2024, 1, 7))
fechas.append(date(2024, 1, 14))
fechas.append(date(2024, 2, 3))
fechas.append(date(2024, 3, 20))

for fecha in fechas:                      # recorrer
    print(fecha.year, fecha.month, fecha.day)

fecha_buscada = date(2024, 1, 14)         # buscar
if fecha_buscada in fechas:
    print("La fecha ya está registrada")
else:
    print("La fecha no está registrada")
```

7.3 Filtrar fechas

```
fechas = [date(2023, 11, 10), date(2024, 1, 7), date(2024, 1, 14),
          date(2024, 2, 3), date(2025, 3, 5)]

# filtrar por año
año_buscado = 2024
for fecha in fechas:
    if fecha.year == año_buscado:
        print(fecha)

# filtrar por mes y año
mes_buscado = 1   # enero
for fecha in fechas:
    if fecha.year == año_buscado and fecha.month == mes_buscado:
        print(fecha)
```

Importante: si no filtras por año, contarás el mismo mes de cualquier año y el resultado será incorrecto.

```
fechas = [date(2023, 12, 25), date(2024, 12, 31),
          date(2024, 11, 20), date(2025, 12, 15)]

# incorrecto: cuenta diciembres de cualquier año
contador = 0
for fecha in fechas:
    if fecha.month == 12:
        contador += 1
print(contador)   # imprime 3

# correcto: solo diciembre de 2024
contador = 0
for fecha in fechas:
    if fecha.year == 2024 and fecha.month == 12:
        contador += 1
print(contador)   # imprime 1
```

7.4 Contar fechas

Contar es filtrar con un acumulador: `contador = contador + 1` cada vez que la fecha cumple la condición.

```
fechas = [date(2024, 1, 7), date(2024, 1, 14), date(2024, 1, 22),
          date(2024, 2, 3), date(2024, 2, 18), date(2024, 3, 20),
          date(2025, 1, 8)]

# contar fechas en un año
año_buscado = 2024
contador = 0
for fecha in fechas:
    if fecha.year == año_buscado:
        contador = contador + 1
print(f"Total en {año_buscado}: {contador}")          # 6

# contar fechas en un mes específico
mes_buscado = 1   # enero
contador = 0
for fecha in fechas:
    if fecha.year == año_buscado and fecha.month == mes_buscado:
        contador = contador + 1
print(f"Total en {mes_buscado}/{año_buscado}: {contador}")   # 3
```

7.5 Contar por mes con un diccionario

Para saber cuántas fechas hay en cada uno de los doce meses de un año, usa un diccionario como contador. Se resuelve en tres pasos.

```
año_buscado = 2024

# paso 1: inicializar el diccionario con los doce meses en cero
dias_por_mes = {1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0,
                7: 0, 8: 0, 9: 0, 10: 0, 11: 0, 12: 0}

# paso 2: recorrer las fechas y acumular por mes
for fecha in fechas:
    if fecha.year == año_buscado:
        mes = fecha.month
        dias_por_mes[mes] = dias_por_mes[mes] + 1

# paso 3: mostrar los resultados
for mes, cantidad in dias_por_mes.items():
    print(f"Mes {mes}: {cantidad} días")
```

Si no quieres inicializar los doce meses, empieza con `conteo = {}` y usa `get()`:

```
conteo = {}
for fecha in fechas:
    mes = fecha.month
    conteo[mes] = conteo.get(mes, 0) + 1

print(conteo.get(3, 0))   # cantidad de fechas de marzo
print(conteo.get(6, 0))   # 0 si no hay fechas de junio, sin error
```

Ojo: este último conteo suma el mismo mes de cualquier año. Si la lista mezcla años, agrega `if fecha.year == año_buscado:` antes de contar, como en 7.3.

8. Separación entre lógica e interfaz

La lógica de negocio (clases y validaciones con `raise`) va en un archivo, y la interacción con el usuario (`print`, `input`, `try/except`) va en otro.

8.1 Estructura de archivos recomendada

```
proyecto/
│
├── logica.py      # clases y lógica de negocio
└── interfaz.py    # programa principal con interacción con el usuario
```

8.2 Contenido de cada archivo

`logica.py`: sin `print()` ni `input()`.

```
from datetime import date

class Empleado:
    def __init__(self, nombre, fecha_nacimiento, salario):
        # validaciones
        if salario <= 0:
            raise ValueError("El salario debe ser positivo")

        # atributos
        self.nombre = nombre
        self.fecha_nacimiento = fecha_nacimiento
        self.salario = salario
        self.historial = []

    def calcular_edad(self, fecha_referencia):
        # aquí va el cálculo de la edad
        return edad

    def cambiar_salario(self, nuevo_salario):
        self.historial.append(self.salario)
        self.salario = nuevo_salario
        return self.salario
```

`interfaz.py`: menú, `input()`, `print()` y captura de errores.

```
from datetime import date
from logica import Empleado

def ingresar_numero(mensaje):
    while True:
        try:
            return float(input(mensaje))
        except ValueError:
            print("Error: debe ingresar un número")

# programa principal con menú, inputs y prints
```

8.3 Reglas de oro

En `logica.py`	En `interfaz.py`
Validaciones con `raise`	Captura de excepciones con `try/except`
Métodos con `return`	`print()` para mostrar resultados
No usar `print()`	`input()` para recibir datos
No usar `input()`	No poner lógica de negocio

8.4 Validar la entrada del usuario en la interfaz

El usuario puede escribir cualquier cosa. La interfaz debe ser robusta y repetir la pregunta en lugar de terminar abruptamente. Todos estos patrones usan `while True` + `try/except` + `return` (sección 5.3).

Número entero

```
def ingresar_entero(nombre_dato: str) -> int:
    """Solicita un entero y no sale del bucle hasta que sea válido."""
    while True:
        try:
            numero = int(input(f"{nombre_dato}: "))
            return numero
        except ValueError:
            print(f"Error: el dato ingresado para {nombre_dato} debe ser un
entero.")
```

Número decimal positivo

```
def ingresar_flotante(mensaje: str) -> float:
    while True:
        try:
            valor = float(input(mensaje))
            if valor <= 0:
                print("Error: el valor debe ser positivo")
                continue
            return valor
        except ValueError:
            print("Error: debe ingresar un número válido")
```

Fecha

```
from datetime import date

def ingresar_fecha() -> date:
    while True:
        try:
            año = int(input("Año: "))
            mes = int(input("Mes: "))
            dia = int(input("Día: "))

            fecha = date(año, mes, dia)    # lanza ValueError si la fecha no
existe

            if fecha > date.today():
                print("Error: la fecha no puede ser futura")
                continue

            return fecha
        except ValueError:
            print("Error: fecha inválida. Intente nuevamente.")
```

Opción de menú

```
while True:
    try:
        opcion = int(input("Seleccione opción: "))
        if 1 <= opcion <= 4:
            break
        else:
            print("Opción fuera de rango")
    except ValueError:
        print("Debe ingresar un número")
```

9. Formato de texto con f-strings

Dentro de las llaves de un f-string, lo que va después de `:` le dice a Python cómo mostrar el valor: miles, decimales, ancho y alineación.

9.1 Números decimales: `:,.2f`

Cada símbolo tiene un papel:

- `:` inicia las especificaciones de formato.
- `,` es el separador de miles (una coma cada tres dígitos).
- `.2` indica exactamente dos decimales.

- `f` indica formato de número flotante.

```
salario = 1500000
print(f"Salario base: {salario:,.2f}")   # Salario base: 1,500,000.00

deuda = 9999999.5
print(f"Deuda total: {deuda:,.2f}")      # Deuda total: 9,999,999.50

porcentaje = 0.2567
print(f"Porcentaje: {porcentaje:.2f}")   # Porcentaje: 0.26 (sin comas)
```

9.2 Alineación de texto: `:<12`

`<` alinea a la izquierda y `12` es el ancho total del campo en caracteres. Así se crean columnas perfectamente alineadas.

```
nombres_meses = {1: "Enero", 2: "Febrero", 3: "Marzo",
                 9: "Septiembre", 12: "Diciembre"}

for mes in [1, 2, 3, 9, 12]:
    print(f"{nombres_meses[mes]:<12}: mes {mes}")
# Enero       : mes 1
# Febrero     : mes 2
# Marzo       : mes 3
# Septiembre  : mes 9
# Diciembre   : mes 12
```

9.3 Otros formatos útiles

```
texto = "hola"
print(f"|{texto:<10}|")   # |hola      |  izquierda
print(f"|{texto:>10}|")   # |      hola|  derecha
print(f"|{texto:^10}|")   # |   hola   |  centrado

codigo = 42
print(f"Código: {codigo:05d}")        # Código: 00042  (relleno con ceros)

proporcion = 0.345
print(f"Progreso: {proporcion:.1%}")  # Progreso: 34.5%  (porcentaje)
```

Formato	Qué hace	Ejemplo de resultado
`:,.2f`	Miles con coma y 2 decimales	`1,500,000.00`

Formato	Qué hace	Ejemplo de resultado
`:.2f`	2 decimales, sin miles	`0.26`
`:<12`	Alinea a la izquierda en 12 espacios	`Enero`
`:>10`	Alinea a la derecha en 10 espacios	`hola`
`:^10`	Centra en 10 espacios	`hola`
`:05d`	Entero con ceros a la izquierda hasta 5 dígitos	`00042`
`:.1%`	Porcentaje con 1 decimal	`34.5%`

10. Guía para diseñar una clase

Para diseñar cualquier clase basta responder cinco preguntas; cada respuesta se convierte en una parte del código.

10.1 Las cinco preguntas

Pregunta	Determina
¿Qué datos necesita este objeto?	Los atributos
¿Qué puede hacer este objeto?	Los métodos
¿Qué datos son obligatorios al crearlo?	Los parámetros de `__init__`
¿Qué debe validarse?	Las condiciones con `raise`
¿Qué debe recordar?	Las listas de historial

10.2 Estructura estándar de una clase

La clase se escribe siempre en este orden: validaciones, atributos, atributos adicionales y luego los métodos.

```
from datetime import date

class NombreClase:
    def __init__(self, parametro1, parametro2):
        # 1. validaciones
        if not isinstance(parametro1, str):
            raise TypeError("mensaje de error")
        if parametro2 < 0:
```

```
            raise ValueError("mensaje de error")

        # 2. asignación de atributos
        self.atributo1 = parametro1
        self.atributo2 = parametro2

        # 3. atributos adicionales
        self.lista_historial = []
        self.fecha_creacion = date.today()

    def metodo_uno(self, parametros):
        # lógica del método
        resultado = self.atributo2 * 2
        return resultado

    def metodo_con_historial(self, nuevo_valor):
        # guardar en el historial
        self.lista_historial.append(self.atributo2)
        # actualizar
        self.atributo2 = nuevo_valor
```

10.3 Flujo de trabajo correcto

1. Definir la clase con sus atributos y métodos.
2. Validar los datos en el constructor.
3. Crear objetos en el programa principal.
4. Llamar métodos para realizar acciones.
5. Capturar errores en la interfaz de usuario.

11. Errores comunes y cómo corregirlos

Los errores de este curso se repiten: casi todos son olvidos de `self`, de validar con `raise`, de proteger el `input()` o de inicializar bien un valor.

Error	Solución	Dónde se explica
Olvidar `self` en un método	El primer parámetro siempre es `self`	2.4
Escribir métodos fuera de la clase	Todo va indentado dentro de `class`	2.1
Olvidar `__init__`	Definir el constructor para inicializar atributos	2.7

Error	Solución	Dónde se explica
`append()` sin decir qué agregar	Pasar el valor: `append(self.monto_actual)`	6.2
Validar con `print`	Usar `raise`, que sí detiene la ejecución	4.1
Convertir `input()` sin proteger	`while True` + `try/except`	8.4
Inicializar `mayor` con 0	Inicializar con el primer elemento	6.4
Contar un mes sin filtrar por año	Filtrar por año **y** mes	7.3
Usar `dic[clave]` en una clave que puede no existir	Usar `dic.get(clave, 0)`	6.6
Mezclar tipos en un `array`	Usar solo valores del tipo declarado (`'i'`, `'d'`)	6.3
Usar un atributo que nunca se inicializó	Crearlo en `__init__`	abajo

Falta `self`

```
# incorrecto
def metodo():            # falta self
    return self.dato

# correcto
def metodo(self):        # self como primer parámetro
    return self.dato
```

`append` sin argumento

```
# incorrecto
self.historial.append()                    # falta qué agregar

# correcto
self.historial.append(self.monto_actual)   # agrega un valor
```

Validar con `print` en lugar de `raise`

```
# incorrecto: solo muestra un mensaje, el programa sigue con el dato malo
if not isinstance(dato, tipo):
    print("error")

# correcto: detiene la ejecución con un error controlado
if not isinstance(dato, tipo):
    raise TypeError("error")
```

`input()` sin proteger

```
# incorrecto: si el usuario escribe texto, el programa se cae
monto = float(input("Monto: "))

# correcto
def ingresar_monto():
    while True:
        try:
            monto = float(input("Monto: "))
            if monto > 0:
                return monto
            print("El monto debe ser positivo")
        except ValueError:
            print("Debe ingresar un número")
```

Atributo que no se inicializó

```
# incorrecto: encendido solo existe después de llamar a encender()
class Carro:
    def __init__(self, marca):
        self.marca = marca

    def encender(self):
        self.encendido = True

    def estado(self):
        if self.encendido:      # AttributeError si nunca se llamó a
encender()
            return "encendido"
        return "apagado"

# correcto: inicializar en __init__
class Carro:
    def __init__(self, marca):
        self.marca = marca
        self.encendido = False
```

```
    def encender(self):
        self.encendido = True

    def estado(self):
        if self.encendido:
            return "encendido"
        return "apagado"
```

12. Conceptos clave, glosario y preguntas de repaso

Si recuerdas la cadena clase → objetos → atributos → métodos → historial, y las frases clave de abajo, tienes la base de todo el curso.

12.1 Analogía: los objetos son como personas

- Nacen con el constructor (`__init__`).
- Tienen datos (atributos): nombre, edad, salario.
- Hacen cosas (métodos): calcular edad, cambiar salario.
- Recuerdan su historia (listas que se llenan solas).
- Su lógica está separada de la interfaz.

12.2 Estructura mental

1. **Clase** (plantilla)
2. **Objetos** (instancias de la plantilla)
3. **Atributos** (datos de cada objeto)
4. **Métodos** (acciones de cada objeto)
5. **Historial** (listas que se llenan con las acciones)

12.3 Frases para recordar

Concepto	Frase clave
Constructor	«El objeto nace aquí»
Atributos	«Los datos del objeto»
Métodos	«Lo que el objeto puede hacer»
`self`	«El objeto a sí mismo»
Listas	«La memoria del objeto»

Concepto	Frase clave
`raise`	«Algo salió mal, detener aquí»
`try/except`	«Intenta esto; si falla, haz esto otro»

12.4 Glosario

Término	Definición	Ejemplo
Clase	Plantilla para crear objetos	`class Empleado:`
Objeto	Instancia concreta de una clase	`laura = Empleado(...)`
Atributo	Variable que pertenece al objeto	`self.nombre`
Método	Función que pertenece a la clase	`def calcular_edad(self):`
Constructor	Método que inicializa el objeto	`def __init__(self):`
`self`	Referencia al objeto actual	`self.salario`
Parámetro	Valor que recibe un método	`def metodo(self, parametro):`
Type hint	Anotación del tipo esperado	`nombre: str`
`raise`	Lanzar una excepción	`raise ValueError("...")`
`try/except`	Capturar y manejar errores	`try: ... except: ...`
Lista	Colección ordenada de elementos, de tipos mezclables	`[1, 2, 3]`
Arreglo	Colección de elementos de un solo tipo, en memoria contigua	`array('i', [1, 2, 3])`
Diccionario	Colección de pares clave → valor	`{1: 5, 2: 3}`
`append`	Agregar un elemento al final de una lista	`lista.append(5)`

12.5 Preguntas de repaso

Responde primero sin mirar la columna de la derecha.

Pregunta	Respuesta
¿Qué diferencia hay entre una clase y un objeto?	La clase es la plantilla; el objeto es una instancia concreta con valores propios en sus atributos.
¿Qué hace `__init__` y cuándo se ejecuta?	Es el constructor: se ejecuta solo al crear el objeto, y sirve para validar datos y crear los atributos.
¿Qué es `self`?	La referencia al objeto con el que se está trabajando; es el primer parámetro de todo método y Python lo pasa solo.
¿Los type hints impiden pasar un tipo incorrecto?	No. Documentan y ayudan al IDE, pero el programa se ejecuta igual. Para exigir el tipo hay que validar con `isinstance` y `raise`.
¿Por qué se valida con `raise` y no con `print`?	`print` solo muestra un mensaje y el programa sigue con el dato malo; `raise` detiene la ejecución con un error controlado.
¿Cuándo se lanza `TypeError` y cuándo `ValueError`?	`TypeError`: el tipo de dato es incorrecto. `ValueError`: el tipo es correcto pero el valor no sirve (negativo, fecha futura).
¿En qué se diferencian `break`, `continue` y `return`?	`break` sale del bucle; `continue` salta a la siguiente vuelta; `return` sale de la función (y de cualquier bucle dentro de ella).
¿Qué debe haber en `logica.py` y qué en `interfaz.py`?	`logica.py`: clases, validaciones con `raise` y métodos con `return`, sin `print` ni `input`. `interfaz.py`: menú, `input`, `print` y `try/except`.
¿Por qué `mayor` se inicializa con el primer elemento y no con 0?	Si todos los valores son negativos, empezar en 0 devolvería 0, que ni siquiera está en el arreglo.
¿Qué diferencia hay entre una lista y un `array`?	La lista puede mezclar tipos; el `array` exige un solo tipo declarado (`'i'` o `'d'`) y no permite mezclar.
¿Para qué sirve `conteo.get(3, 0)`?	Devuelve el valor de la clave 3, o 0 si no existe, sin lanzar `KeyError`.

Pregunta	Respuesta
¿Por qué contar diciembres exige filtrar también por año?	Porque sin el año se cuentan los diciembres de cualquier año y el resultado es incorrecto.
¿Qué imprime `print(f"{1500000:,.2f}")`?	`1,500,000.00`